

# Evaluación 4

**Nombres: Nicolas benavides, Eidan novoa**

**Carrera: Desarrollo de videojuegos**

**Docente: Cristian Andrés Peña Villar**

**Módulo: programación 3**

**Fecha: 19/06/2026**

## 1. Contextualización

En la etapa anterior se definió la lógica base de personajes, atributos básicos y estructura inicial básica y estructura inicial de clases; se detectaron acoplamientos fuertes entre sistemas (Combate-interacción-inventario) y falta de abstracción. Se reutilizaron:

- Atributos generales: Vida, nombre, daño
- Esqueleto de clases base
- Se modificó: Jerarquía de herencia, se agregaron interfaces y manejo de eventos para desacoplar.

## Resumen gráfico:

- Personaje (abstracta): +Vida, +Nombre, +RecibirDaño()
- Enemigo : Personaje + Inventario, +Movimiento
- Jugador : Personaje +IA, +TipoEnemigo
- IDamageable = RecibirDaño (float)
- IInteractable = Interactuar(Personaje)
- Inventario : Lista generica + metodos Agregar/Quitar/Usar

## 2. Implementación técnica

### Personajes - Jerarquía de herencia

Lógica: Herencia simple + Simple + polimorfismo evita duplicación; Personaje es la clase subclases especializan.

**Interfaces:** IDamageable e IInteractable

**Propósito:** Definir Contratos sin imponer herencia; Cualquier objeto que implemente puede ser dañado o interactuado

**Aplicación:** Jugador , Enemigo , Cofre , PlantaDañable comparten comportamiento sin pertenecer a la misma rama. Facilita extensión: nueva entidad solo implementa la interfaz.

### inventario y eventos

Ventaja eventos: Desacopla inventario de UI, sonido, logros “Conoce” quien reacciona solo avisa. Menos errores, más fácil probar y modificar.

Ejemplo namespace:

```
namespace JuegosMecanicos.Personajes
{
    public class Jugador: personaje ()
}
```

### 3. Diagnóstico de rendimiento - Unity Profiler

(Insertar Capturas: Escena + Profiler, una por fase: Primitivas / Low Poly/ High poly)

Componentes: gpu, gtx 1660 super, cpu: ryzen 7 5700g, ram: 16gb

#### Escena 1

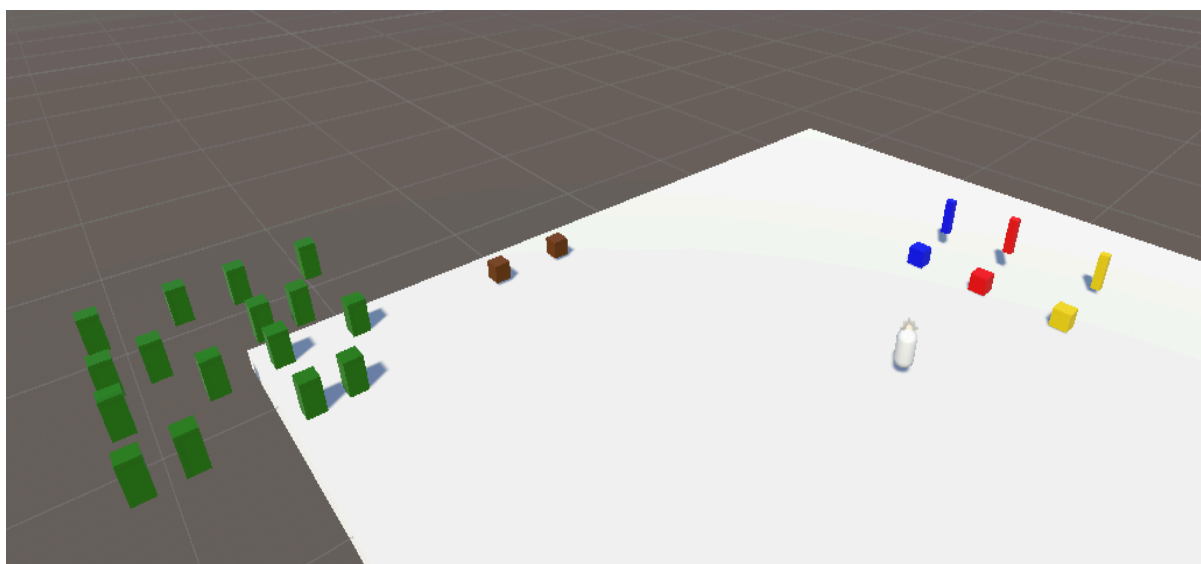


Tabla 1 – Métricas de rendimiento

| información     | Datos                       |
|-----------------|-----------------------------|
| FPS promedio    | 900 - 1000 FPS              |
| Tiempo CPU (ms) | 1.0 ms y 0.2 de rendimiento |

Tabla 2 - Conteo de objetos y polígonos

| Objeto | Nombre asignado | Triángulos aprox    | Cantidad    | Total Triángulos |
|--------|-----------------|---------------------|-------------|------------------|
| Cubo   | Cubo (Color)    | 12 triángulos aprox | 24 Unidades | 228              |

## Escena 2:

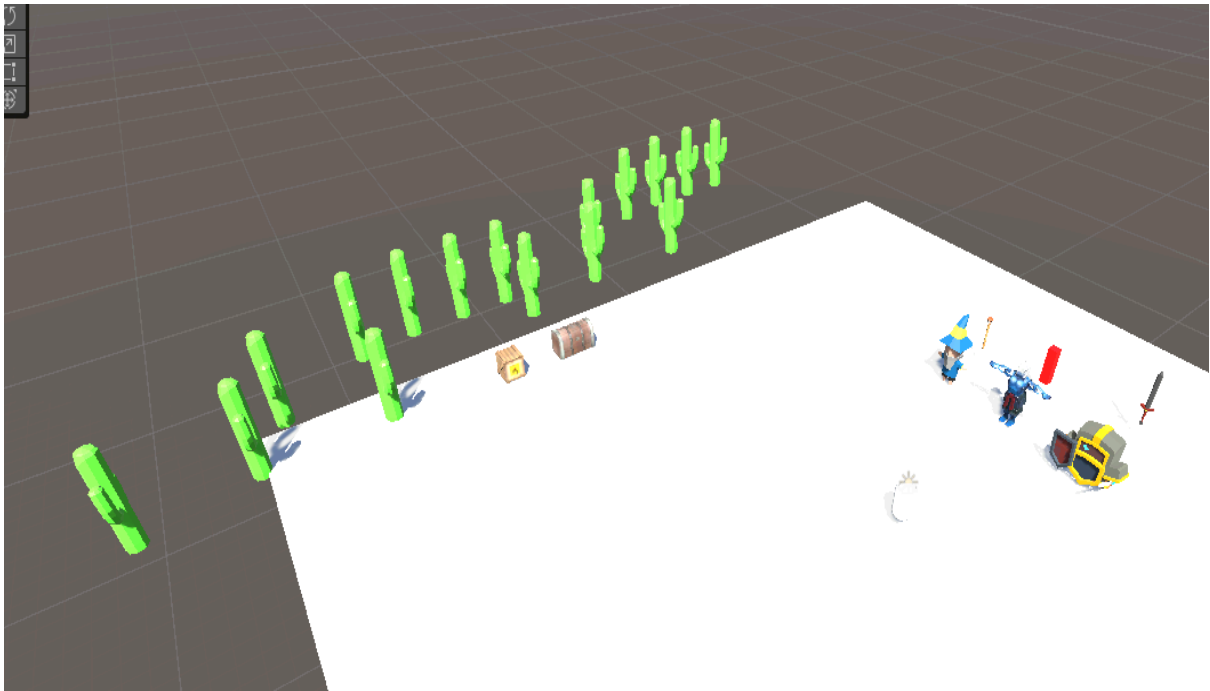


Tabla 1 – Métricas de rendimiento

| información     | Datos                       |
|-----------------|-----------------------------|
| FPS promedio    | 800 - 960 FPS               |
| Tiempo CPU (ms) | 1.3 ms y 0.4 de rendimiento |

Tabla 2 - Conteo de objetos y polígonos

| Tipo Objeto | Nombre asignado | Triángulos aprox      | Cantidad | Total Triángulos |
|-------------|-----------------|-----------------------|----------|------------------|
| Cubo        | Cubo rojo       | 12 triángulos aprox   | 1 Objeto | 12 Triángulos    |
| Mesh        | Paladín         | 1630 triángulos aprox | 1 Objeto | 1630 Triángulos  |
| Mesh        | Mago            | 376 triángulos aprox  | 1 Objeto | 376 Triángulos   |
| Mesh        | Luchador        | 1404 triángulos aprox | 1 Objeto | 1404 Triángulos  |

|      |        |                        |            |                 |
|------|--------|------------------------|------------|-----------------|
| Mesh | Caja   | 72 triángulos<br>aprox | 1 Objeto   | 72 Triángulos   |
| Mesh | Cofre  | 350 triángulos         | 1 Objeto   | 350 Triángulos  |
| Mesh | Cactus | 416 Triángulos         | 16 Objeto  | 6,656Triángulos |
| Mesh | Espada | 122 Triángulos         | 1 Objeto   | 122 Triángulos  |
| Mesh | Varita | 718 Triángulos         | 1 Objeto   | 718 Triángulos  |
|      |        | Total                  | 24 Objetos | 11,340          |

## Escena 3 :

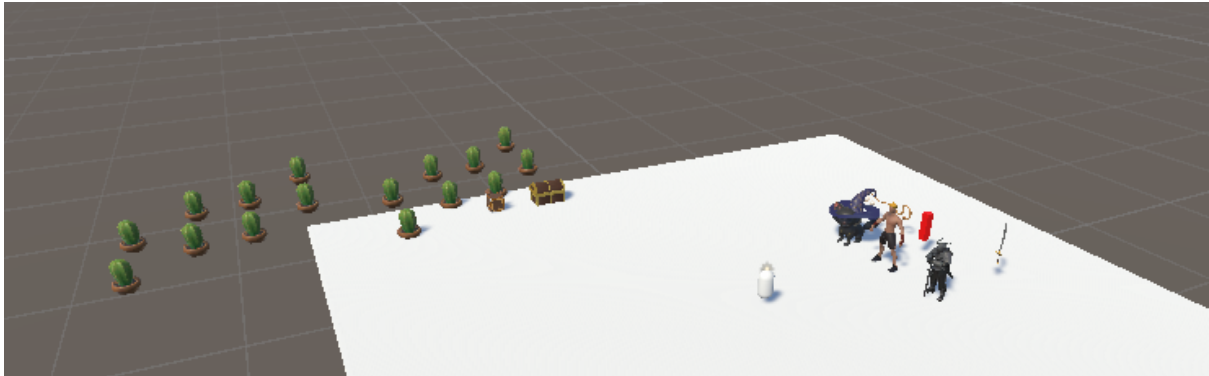


Tabla 1 – Métricas de rendimiento

| información     | Datos                       |
|-----------------|-----------------------------|
| FPS promedio    | 700 - 900 FPS               |
| Tiempo CPU (ms) | 1.7 ms y 0.4 de rendimiento |

Tabla 2 - Conteo de objetos y polígonos

| Objeto | Nombre asignado  | Triángulos aprox        | Cantidad | Triangulos triangulos |
|--------|------------------|-------------------------|----------|-----------------------|
| Cubo   | Cubo Rojo (1)    | 12 triángulos aprox     | 1 Objeto | 12 triángulos         |
| Mesh   | Paladín          | 16,840 triángulos aprox | 1 Objeto | 16,840 triángulos     |
| Mesh   | Mago             | 10,840 triángulos aprox | 1 Objeto | 10,840 triángulos     |
| Mesh   | Luchador         | 56,140 triángulos aprox | 1 Objeto | 56,140 triángulos     |
| Cofre  | Cofrecito        | 1,904                   | 1 Objeto | 1,904 triángulos      |
| espada | Cubo Amarillo(1) | 1,129,330               | 1 Objeto | 1,129,330 triangulos  |
| Varita | Cubo Azul (1)    | 10,110 triángulos       | 1 Objeto | 10110 triángulos      |

|        |        |                         |            |                         |
|--------|--------|-------------------------|------------|-------------------------|
|        |        | aprox                   |            |                         |
| Cactus | Cactus | 540 triángulos<br>aprox | 16 Objetos | 8,640 triángulos        |
| Caja   | Caja   | 1146<br>triángulos      | 1 Objeto   | 1,146 triángulos        |
|        |        | Total                   | 24 objetos | 1,234,962<br>triangulos |

## Análisis técnico

- Mejor rendimiento: Fase Primitivas = carga geométrica mínima, GPU y CPU procesan rápido, sin cuellos de botella.
- Peor rendimiento: Fase high Poly = cantidad de triángulos 10-20x mayor, mayor consumo de memoria VRAM y tiempo de procesamiento por lote de dibujo.
- Relación directa: A mayor número de triángulos, mayor tiempo por frame y menor FPS, El hardware (CPU/GPU) no varía, por lo que la complejidad visual es el factor determinante.



## 4. Conclusión

El uso de herencia reduce repetición y centraliza cambios, las interfaces permiten combinar comportamientos flexibles sin rigidez. los eventos separan responsabilidades y facilitan el mantenimiento.

En rendimiento, la optimización no es opcional, la escalabilidad depende de limitar la carga geométrica y una estructura clara permite agregar contenido sin colapsar el rendimiento - fundamental para proyectos profesionales.